APPENDIX A: Protocol for Semantic-Theme Bibliometric Mapping (STBM)Analysis: A Replicable Step-by-Step Methodology for Bibliometric Research

© José Rafael Caro-Barrera

Abstract

By following this protocol, researchers can produce comprehensive, high-quality bibliometric analyses suitable for any field while generating actionable insights for practical and policy implications.

Protocol Version: 1.0

Last Updated: 27th, January 2026

Contact: z52cabaj@uco.es

Available at: https://github.com/jrcarob/astrotourism and https://osf.io/t3gfw/overview

Introduction

This protocol provides a complete, replicable methodology for conducting Semantic-Theme Bibliometric Mapping (STBM) analysis. The STBM approach integrates three analytical dimensions: (1) traditional bibliometric indicators (performance analysis), (2) thematic mapping through co-word analysis and strategic diagrams, and (3) semantic field identification revealing underlying paradigms and meaning structures.

Key Software Requirements:

- R® (version 4.0 or higher) with RStudio®
- bibliometrix® R package (version 4.0+)
- VOSviewer® (version 1.6.18+)
- Microsoft® Excel® or equivalent
- Optional: Gephi® (for advanced network visualization)

Estimated Time: 40-60 hours for complete analysis (excluding literature interpretation)

Skill Level Required: Intermediate (basic R programming, understanding of bibliometric concepts)

STAGE 1: DATA RETRIEVAL FROM MULTIPLE DATABASES

1.1 Web of Science Core Collection

Access: Institutional subscription required (http://webofscience.com)

Step-by-Step Search Process:

1. **Navigate to Web of Science Core Collection**
   - Log in through institutional access
   - Select "Web of Science Core Collection" (not "All Databases")
   - Recommended: Include all citation indexes (SCI-EXPANDED, SSCI, A&HCI, ESCI)

2. **Construct Search Query**

```
Basic Search Formula:
TS=("astrotourism" OR "astronomical tourism" OR "dark sky tourism"
    OR "starlight tourism" OR "celestial tourism" OR "dark-sky tourism"
    OR "astro-tourism" OR "astronomy tourism")

AND

TS=(tourism OR tourist* OR destination* OR travel*)
```

   Field Codes:

   - TS = Topic (searches Title, Abstract, Author Keywords, Keywords Plus)
   - TI = Title only
   - AB = Abstract only
   - AK = Author Keywords only

   Search Strategy Rationale:

   - Include variant spellings (astrotourism, astro-tourism, dark-sky, dark sky)
   - Include synonyms (astronomical tourism, celestial tourism, starlight tourism)
   - Second line ensures tourism focus (excludes pure astronomy papers)
   - Use truncation (*) for word variants (tourist, tourists, tourism)

3. **Apply Filters**

   - **Timespan:** 2006-2024 (or ALL YEARS for comprehensive coverage)
   - **Document Types:** Articles, Reviews, Book Chapters, Conference Papers
   - **Language:** All languages (or specify if needed)
   - **Exclude:** Corrections, Editorial Material, News Items

4. **Review Results**

   - Check total number of results (expect 150-250 for astrotourism)
   - Scan first 20 results to verify relevance
   - If too many irrelevant results, refine search
   - If too few results, consider broadening terms

5. **Export Data**

   CRITICAL: Export in Multiple Formats

   Format 1: Plain Text (for bibliometrix)

   - Click "Export"
   - Select "Plain Text"
   - Records: Select "Full Record and Cited References"
   - Number of Records: 500 maximum per export (if more, do multiple exports)
   - File name: `WoS_Astrotourism_Records1-500.txt`
   - Click "Export"

   Format 2: Tab-delimited (for backup/Excel)

   - Export same records
   - Select "Tab-delimited (Win, UTF-8)"
   - Records: "Full Record"
   - Save as: `WoS_Astrotourism_Records_Excel.txt`

   Format 3: BibTeX (for reference management)

   - Export same records
   - Select "BibTeX"
   - Save as: `WoS_Astrotourism.bib`

6. **Document Search Details**

   - Save search history (WoS allows saving searches)
   - Screenshot or copy the exact search query
   - Record: Date of search, number of results, filters applied
   - Create file: `WoS_Search_Documentation.txt`

1.2 Scopus Database

Access: Institutional subscription required (http://scopus.com)

Step-by-Step Search Process:

1. **Navigate to Scopus Advanced Search**
   - Log in through institutional access
   - Click "Advanced" search option
2. **Construct Search Query**

```
Advanced Search Formula:
TITLE-ABS-KEY("astrotourism" OR "astronomical tourism" OR
               "dark sky tourism" OR "starlight tourism" OR
               "celestial tourism" OR "dark-sky tourism" OR
               "astro-tourism" OR "astronomy tourism")

AND

TITLE-ABS-KEY(tourism OR tourist* OR destination* OR travel*)
```

   Field Codes:

   - TITLE-ABS-KEY = Searches title, abstract, and author keywords
   - ALL = Searches all fields (too broad, not recommended)
   - TITLE = Title only
   - KEY = Author keywords only
3. **Apply Filters (Left Sidebar)**
   - **Date Range:** 2006-2024 (or customize)
   - **Document Type:** Article, Review, Book Chapter, Conference Paper
   - **Source Type:** Journals, Books, Conference Proceedings
   - **Language:** All or specify
   - **Publication Stage:** Final only (exclude "Article in Press" if needed)
4. **Verify Results**
   - Scopus typically returns 10-20% more results than WoS (more source coverage)
   - Check for relevance by scanning top results
   - Overlap with WoS expected: 60-80%
5. **Export Data**

   IMPORTANT: Scopus limits exports to 2,000 records

   Export Format 1: CSV (for bibliometrix)

   - Select all documents (check box at top)
   - Click "Export"
   - Choose "CSV" format

- Information: Select "Citation information" and "Abstract & keywords"
- Click "Export"
- Save as: `Scopus_Astrotourism.csv`

Export Format 2: BibTeX

- Select all documents
- Export → BibTeX
- Save as: `Scopus_Astrotourism.bib`

Export Format 3: RIS (for reference managers)

- Select all documents
- Export → RIS
- Save as: `Scopus_Astrotourism.ris`

6. **Document Search**
   - Save search query (Scopus allows saving with account)
   - Create documentation file: `Scopus_Search_Documentation.txt`
   - Record: Search string, date, filters, number of results

1.3 OpenAlex Database

Access: Free and open (https://openalex.org)

Step-by-Step Search Process:

1. **Navigate to OpenAlex Website**
   - Go to https://openalex.org
   - Click "Search Works"
2. **Web Interface Search (Limited)**

   ```
   Basic Search:
   astrotourism OR "dark sky tourism" OR "astronomical tourism"
   ```

   Limitation: Web interface doesn't support complex Boolean queries well
   Solution: Use API or downloaded snapshot for complex searches
3. **API Search (Recommended for Comprehensive Retrieval)**

   Option A: Use OpenAlex API (Requires Programming)

   Install Python library:

```
pip install pyalex
```

Python script for retrieval:

```
from pyalex import Works
import pandas as pd
```

```
# Configure email for polite API usage
from pyalex import config
config.email = "your.email@institution.edu"

# Search query
results = Works().search("astrotourism OR astronomical tourism OR dark sky tourism").get(

# Convert to dataframe
df = pd.DataFrame(results)

# Export to CSV
df.to_csv("OpenAlex_Astrotourism.csv", index=False)
```

Option B: Use R with openalexR package

```
#| output: false
# Install package
install.packages("openalexR")
library(openalexR)

# Search OpenAlex
results <- oa_fetch(
  entity = "works",
  search = "astrotourism OR astronomical tourism OR dark sky
  tourism",
  from_publication_date = "2006-01-01",
  to_publication_date = "2024-12-31",
  options = list(select = c("id", "title", "authorships",
                            "publication_date",
                            "cited_by_count",
                            "keywords",
                            "abstract"))
)

# Save results
write.csv(results, "OpenAlex_Astrotourism.csv", row.names = FALSE)
```

4. **Export Considerations**
   - OpenAlex provides free, comprehensive coverage (might find 20-30% more items than Scopus)
   - Data quality variable (some records incomplete)
   - Excellent for open science and ensuring comprehensive coverage
   - Export format already compatible with R
5. **Document Search**
   - Save API query code
   - Record: Date, query, number of results

- File: `OpenAlex_Search_Documentation.txt`

1.4 Search Documentation Template

Create a master documentation file: `Search_Protocol_Master.txt`

```
BIBLIOMETRIC SEARCH DOCUMENTATION
Research Topic: [e.g., Astrotourism]
Researcher: [Name]
Date of Search: [YYYY-MM-DD]

================================================================================
WEB OF SCIENCE CORE COLLECTION
================================================================================
Date Searched: [YYYY-MM-DD]
Search Query:
TS=("astrotourism" OR "astronomical tourism" OR "dark sky tourism"
    OR "starlight tourism" OR "celestial tourism")
AND
TS=(tourism OR tourist* OR destination*)

Timespan: 2006-2024
Indexes: SCI-EXPANDED, SSCI, A&HCI, ESCI
Document Types: Articles, Reviews, Book Chapters, Conference Papers
Language: All
Results: [Number] documents
Export Files:
  - WoS_Astrotourism_Records1-500.txt
  - WoS_Astrotourism_Records_Excel.txt

================================================================================
SCOPUS
================================================================================
Date Searched: [YYYY-MM-DD]
Search Query:
TITLE-ABS-KEY("astrotourism" OR "astronomical tourism" OR "dark sky tourism")
AND
TITLE-ABS-KEY(tourism OR tourist* OR destination*)

Date Range: 2006-2024
Document Types: Article, Review, Book Chapter, Conference Paper
Language: All
Results: [Number] documents
Export Files:
  - Scopus_Astrotourism.csv
  - Scopus_Astrotourism.bib
```

```
================================================================================
OPENALEX
================================================================================
Date Searched: [YYYY-MM-DD]
Search Method: API via openalexR
Search Query: [Copy R/Python code]
Date Range: 2006-01-01 to 2024-12-31
Results: [Number] documents
Export Files:
  - OpenAlex_Astrotourism.csv

================================================================================
TOTAL RESULTS BEFORE DEDUPLICATION
================================================================================
Web of Science: [Number]
Scopus: [Number]
OpenAlex: [Number]
Total Retrieved: [Sum]
Expected Duplicates: ~40-60%
Expected Unique After Merging: [Estimate]

================================================================================
NOTES
================================================================================
[Any observations, challenges, or decisions made during search process]
```

STAGE 2: DATA MERGING AND DEDUPLICATION IN R

2.1 Setup R Environment

Install Required Packages:

```
#| output: false
# Install packages (run once)
install.packages("bibliometrix")
install.packages("dplyr")
install.packages("tidyr")
install.packages("stringr")
install.packages("openalexR")

# Load libraries (run each session)
library(bibliometrix)
library(dplyr)
library(tidyr)
```

```
library(stringr)
```

2.2 Import Data from Each Database

Create R Project Structure:

```
Project_Folder/
|-- 01_Raw_Data/
|   |-- WoS_Astrotourism_Records1-500.txt
|   |-- Scopus_Astrotourism.csv
|   |-- OpenAlex_Astrotourism.csv
|-- 02_Processed_Data/
|-- 03_Analysis/
|-- 04_Results/
|-- Scripts/
    |-- 01_data_Import.R
    |-- 02_deduplication.R
    |-- 03_analysis.R
```

Script: 01_Data_Import.R

```
#| output: false
# Set working directory
setwd("~/Your_Project_Folder")

# Import Web of Science data
wos_data <- convert2df(
  file = "01_Raw_Data/WoS_Astrotourism_Records1-500.txt",
  dbsource = "wos",
  format = "plaintext"
)

# Check import
cat("WoS records imported:", nrow(wos_data), "\n")
cat("WoS columns:", ncol(wos_data), "\n")
head(wos_data[, c("TI", "AU", "PY", "SO")], 3)

# Import Scopus data
scopus_data <- convert2df(
  file = "01_Raw_Data/Scopus_Astrotourism.csv",
  dbsource = "scopus",
  format = "csv"
)

# Check import
cat("Scopus records imported:", nrow(scopus_data), "\n")
cat("Scopus columns:", ncol(scopus_data), "\n")
```

```
head(scopus_data[, c("TI", "AU", "PY", "SO")], 3)

# Import OpenAlex data (already in R-friendly format)
openalex_data <- read.csv("01_Raw_Data/OpenAlex_Astrotourism.csv")

# Convert OpenAlex to bibliometrix format
# Note: May require custom conversion depending on OpenAlex export structure
# This is a simplified example - adjust field names as needed
openalex_converted <- data.frame(
  TI = openalex_data$title,
  AU = openalex_data$author_names,  # May need parsing
  PY = substr(openalex_data$publication_date, 1, 4),
  SO = openalex_data$source,
  DI = openalex_data$doi,
  AB = openalex_data$abstract,
  TC = openalex_data$cited_by_count
)

cat("OpenAlex records imported:", nrow(openalex_converted), "\n")

# Save imported data for backup
save(wos_data, scopus_data, openalex_converted,
     file = "02_Processed_Data/imported_data.RData")
```

2.3 Standardize Column Names Across Databases

Challenge: Different databases use different field names

Solution: Create standardized field mapping

```
#| output: false
# Function to standardize column names
standardize_columns <- function(df, source) {

  # Create mapping based on source
  if(source == "wos") {
    # WoS already in bibliometrix standard format
    return(df)
  }

  if(source == "scopus") {
    # Scopus already in bibliometrix standard format
    return(df)
  }

  if(source == "openalex") {
```

```
    # Custom mapping for OpenAlex
    # Adjust based on your actual OpenAlex export
    standard_names <- c(
      "TI" = "title",
      "AU" = "authors",
      "PY" = "year",
      "SO" = "source",
      "DI" = "doi",
      "AB" = "abstract",
      "TC" = "citations"
    )

    # Rename columns
    for(new_name in names(standard_names)) {
      old_name <- standard_names[new_name]
      if(old_name %in% names(df)) {
        names(df)[names(df) == old_name] <- new_name
      }
    }

    return(df)
  }
}

# Apply standardization
wos_std <- standardize_columns(wos_data, "wos")
scopus_std <- standardize_columns(scopus_data, "scopus")
openalex_std <- standardize_columns(openalex_converted, "openalex")

# Add database source column for tracking
wos_std$DB_SOURCE <- "WoS"
scopus_std$DB_SOURCE <- "Scopus"
openalex_std$DB_SOURCE <- "OpenAlex"
```

2.4 Merge Databases

Script: 02_Deduplication.R

```
#| output: false
# Merge all databases
# bibliometrix has a built-in merge function

merged_data <- mergeDbSources(
  wos_std,
  scopus_std,
  openalex_std,
```

```
  remove.duplicated = TRUE
)

# Check merge results
cat("Total records before deduplication:",
    nrow(wos_std) + nrow(scopus_std) + nrow(openalex_std), "\n")
cat("Total records after automatic deduplication:",
    nrow(merged_data), "\n")
cat("Duplicates removed:",
    (nrow(wos_std) + nrow(scopus_std) + nrow(openalex_std)) - nrow(merged_data), "\n")

# Save merged data
write.csv(merged_data, "02_Processed_Data/merged_data_auto.csv", row.names = FALSE)
save(merged_data, file = "02_Processed_Data/merged_data_auto.RData")
```

2.5 Advanced Deduplication (Manual Check)

Automated deduplication may miss variants

Create comprehensive deduplication function:

```
#| output: false
# Advanced deduplication function
advanced_deduplication <- function(df) {

  # Create matching keys based on multiple fields
  df <- df %>%
    mutate(
      # Normalize titles (lowercase, remove punctuation)
      TI_clean = str_to_lower(TI) %>%
        str_replace_all("[[:punct:]]", "") %>%
        str_squish(),

      # Extract first author last name
      AU_first = str_extract(AU, "^[A-Z]+"),

      # Create composite key
      match_key = paste(TI_clean, AU_first, PY, sep = "_")
    )

  # Identify duplicates
  df <- df %>%
    group_by(match_key) %>%
    mutate(
      duplicate_group = cur_group_id(),
      is_duplicate = n() > 1,
```

```
      duplicate_count = n()
    ) %>%
    ungroup()

  # For duplicates, keep the record with most complete information
  df_dedup <- df %>%
    group_by(match_key) %>%
    arrange(desc(!is.na(DI)), desc(!is.na(AB)), desc(!is.na(TC))) %>%
    slice(1) %>%
    ungroup()

  # Report
  cat("Records before advanced deduplication:", nrow(df), "\n")
  cat("Duplicate groups found:", max(df$duplicate_group[df$is_duplicate]), "\n")
  cat("Records after advanced deduplication:", nrow(df_dedup), "\n")
  cat("Additional duplicates removed:", nrow(df) - nrow(df_dedup), "\n")

  # Create duplicate report for manual inspection
  duplicates_report <- df %>%
    filter(is_duplicate) %>%
    arrange(duplicate_group, DB_SOURCE) %>%
    select(duplicate_group, TI, AU, PY, SO, DB_SOURCE, DI, TC)

  write.csv(duplicates_report,
            "02_Processed_Data/duplicates_report.csv",
            row.names = FALSE)

  return(df_dedup)
}

# Apply advanced deduplication
final_data <- advanced_deduplication(merged_data)

# Save final deduplicated dataset
write.csv(final_data, "02_Processed_Data/final_dataset.csv", row.names = FALSE)
save(final_data, file = "02_Processed_Data/final_dataset.RData")

# Create summary statistics
summary_stats <- data.frame(
  Database = c("Web of Science", "Scopus", "OpenAlex", "Merged (Auto)", "Final (Manual)"),
  Records = c(nrow(wos_std), nrow(scopus_std), nrow(openalex_std),
              nrow(merged_data), nrow(final_data)),
  Percentage = c(
    round(100 * nrow(wos_std) / nrow(wos_std), 1),
    round(100 * nrow(scopus_std) / nrow(wos_std), 1),
```

```
    round(100 * nrow(openalex_std) / nrow(wos_std), 1),
    round(100 * nrow(merged_data) / nrow(wos_std), 1),
    round(100 * nrow(final_data) / nrow(wos_std), 1)
  )
)

print(summary_stats)
write.csv(summary_stats, "02_Processed_Data/deduplication_summary.csv", row.names = FALSE)
```

2.6 Data Quality Check and Cleaning

```
#| output: false
# Data quality assessment
quality_check <- function(df) {

  cat("\n=== DATA QUALITY REPORT ===\n\n")

  # Total records
  cat("Total records:", nrow(df), "\n\n")

  # Completeness by field
  completeness <- data.frame(
    Field = c("Title", "Authors", "Year", "Source", "DOI", "Abstract",
              "Keywords", "Citations", "References"),
    Column = c("TI", "AU", "PY", "SO", "DI", "AB", "DE", "TC", "CR"),
    Complete = NA,
    Missing = NA,
    Percent_Complete = NA
  )

  for(i in 1:nrow(completeness)) {
    col <- completeness$Column[i]
    if(col %in% names(df)) {
      complete <- sum(!is.na(df[[col]]) & df[[col]] != "")
      missing <- nrow(df) - complete
      completeness$Complete[i] <- complete
      completeness$Missing[i] <- missing
      completeness$Percent_Complete[i] <- round(100 * complete / nrow(df), 1)
    } else {
      completeness$Complete[i] <- 0
      completeness$Missing[i] <- nrow(df)
      completeness$Percent_Complete[i] <- 0
    }
  }
```

```
  print(completeness)

  # Temporal coverage
  cat("\n=== TEMPORAL COVERAGE ===\n")
  if("PY" %in% names(df)) {
    year_dist <- table(df$PY)
    cat("Year range:", min(df$PY, na.rm = TRUE), "-", max(df$PY, na.rm = TRUE), "\n")
    cat("Most productive year:", names(which.max(year_dist)),
        "(", max(year_dist), "documents )\n")
  }

  # Document types
  cat("\n=== DOCUMENT TYPES ===\n")
  if("DT" %in% names(df)) {
    print(table(df$DT))
  }

  # Language distribution
  cat("\n=== LANGUAGE DISTRIBUTION ===\n")
  if("LA" %in% names(df)) {
    print(head(sort(table(df$LA), decreasing = TRUE), 10))
  }

  # Save quality report
  write.csv(completeness, "02_Processed_Data/data_quality_report.csv", row.names = FALSE)

  return(completeness)
}

# Run quality check
quality_report <- quality_check(final_data)

# Clean data based on quality check
final_data_clean <- final_data %>%
  filter(
    !is.na(TI),           # Must have title
    !is.na(AU),           # Must have authors
    !is.na(PY),           # Must have year
    PY >= 2006,           # Within timespan
    PY <= 2024
  )

cat("\nRecords removed due to missing critical fields:",
    nrow(final_data) - nrow(final_data_clean), "\n")
```

```
# Save final clean dataset
save(final_data_clean, file = "02_Processed_Data/final_dataset_clean.RData")
write.csv(final_data_clean, "02_Processed_Data/final_dataset_clean.csv", row.names = FALSE)
```

2.7 Export for VOSviewer

VOSviewer requires specific format

```
#| output: false
# Export for VOSviewer co-citation analysis
# VOSviewer accepts various formats, RIS is most reliable

# Convert to RIS format
convert2df(
  final_data_clean,
  dbsource = "generic",
  format = "csv"
) %>%
  df2bibliometrix() %>%
  write_delim("02_Processed_Data/for_vosviewer.ris", delim = "\n")

# Alternative: Export as tab-delimited for VOSviewer
vosviewer_export <- final_data_clean %>%
  select(TI, AU, PY, SO, DI, AB, DE, ID, CR, TC) %>%
  mutate(
    # VOSviewer-friendly author format (semicolon-separated)
    AU = str_replace_all(AU, ";", "; ")
  )

write.table(vosviewer_export,
            "02_Processed_Data/for_vosviewer.txt",
            sep = "\t",
            row.names = FALSE,
            quote = FALSE)

cat("\nFiles exported for VOSviewer:\n")
cat("- for_vosviewer.ris\n")
cat("- for_vosviewer.txt\n")
```

PHASE 3: BIBLIOMETRIC ANALYSIS IN R (bibliometrix)

3.1 Load Final Dataset

Script: 03_Analysis.R

```
#| output: false
# Load libraries
library(bibliometrix)
library(dplyr)
library(ggplot2)

# Load final clean dataset
load("02_Processed_Data/final_dataset_clean.RData")

# Create bibliometric object
M <- final_data_clean

# Basic info
cat("Dataset loaded successfully\n")
cat("Total documents:", nrow(M), "\n")
cat("Timespan:", min(M$PY), "-", max(M$PY), "\n")
```

3.2 Descriptive Statistics (Main Information)

```
#| output: false
# Generate main information table
results <- biblioAnalysis(M, sep = ";")

# Summary
summary_results <- summary(results, k = 10, pause = FALSE)

# Create custom main information table
main_info <- data.frame(
  Description = c(
    "DOCUMENTS",
    "TIMESPAN",
    "SOURCES (Journals, Books, etc.)",
    "AVERAGE YEARS FROM PUBLICATION",
    "AVERAGE CITATIONS PER DOCUMENT",
    "AVERAGE CITATIONS PER YEAR PER DOC",
    "REFERENCES",
    "",
    "DOCUMENT TYPES",
    "ARTICLE",
    "BOOK",
    "BOOK CHAPTER",
    "CONFERENCE PAPER",
    "REVIEW",
    "",
    "DOCUMENT CONTENTS",
```

```
    "KEYWORDS PLUS (ID)",
    "AUTHOR'S KEYWORDS (DE)",
    "",
    "AUTHORS",
    "AUTHORS",
    "AUTHOR APPEARANCES",
    "AUTHORS OF SINGLE-AUTHORED DOCUMENTS",
    "AUTHORS OF MULTI-AUTHORED DOCUMENTS",
    "",
    "AUTHORS COLLABORATION",
    "SINGLE-AUTHORED DOCUMENTS",
    "DOCUMENTS PER AUTHOR",
    "AUTHORS PER DOCUMENT",
    "CO-AUTHORS PER DOCUMENT",
    "COLLABORATION INDEX",
    "INTERNATIONAL CO-AUTHORSHIPS %"
  ),
  Results = ""
)

# Calculate statistics
main_info$Results[main_info$Description == "DOCUMENTS"] <- nrow(M)
main_info$Results[main_info$Description == "TIMESPAN"] <-
  paste0(min(M$PY, na.rm=TRUE), ":", max(M$PY, na.rm=TRUE))
main_info$Results[main_info$Description == "SOURCES (Journals, Books, etc.)"] <-
  length(unique(M$SO))

# Continue filling other statistics...
# (Full calculation code would be extensive, this shows structure)

# Save main information table
write.csv(main_info, "04_Results/01_Main_Information.csv", row.names = FALSE)

# Generate automatic report with bibliometrix
options(width = 130)
sink("04_Results/bibliometrix_report.txt")
print(summary_results)
sink()
```

3.3 Annual Scientific Production

```
#| output: false
# Calculate annual production
annual_production <- M %>%
  group_by(PY) %>%
```

```
  summarise(Articles = n()) %>%
  arrange(PY)

# Calculate cumulative production
annual_production <- annual_production %>%
  mutate(Cumulative = cumsum(Articles))

# Calculate growth rate
annual_production <- annual_production %>%
  mutate(
    Growth_Rate = (Articles / lag(Articles) - 1) * 100,
    Annual_Growth_Rate_Percent = round(Growth_Rate, 2)
  )

# Save results
write.csv(annual_production,
          "04_Results/02_Annual_Scientific_Production.csv",
          row.names = FALSE)

# Visualize
ggplot(annual_production, aes(x = PY, y = Articles)) +
  geom_area(fill = "steelblue", alpha = 0.7) +
  geom_line(color = "darkblue", size = 1) +
  geom_point(color = "darkblue", size = 2) +
  labs(title = "Annual Scientific Production",
       x = "Year",
       y = "Number of Articles") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, size = 14, face = "bold"))

ggsave("04_Results/annual_production_plot.png", width = 10, height = 6, dpi = 300)
```

3.4 Most Relevant Sources

```
#| output: false
# Calculate source metrics
source_analysis <- M %>%
  group_by(SO) %>%
  summarise(
    Articles = n(),
    Total_Citations = sum(TC, na.rm = TRUE),
    Avg_Citations = round(mean(TC, na.rm = TRUE), 2)
  ) %>%
  arrange(desc(Articles)) %>%
  mutate(Rank = row_number())
```

```
# Top 20 sources
top_sources <- head(source_analysis, 20)

write.csv(top_sources,
          "04_Results/03_Most_Relevant_Sources.csv",
          row.names = FALSE)

# Calculate h-index for sources
source_h_index <- Hindex(M, field = "SO", elements = top_sources$SO, sep = ";", years = Inf)

# Merge h-index with source analysis
source_metrics <- merge(top_sources,
                        source_h_index$H,
                        by.x = "SO",
                        by.y = "Element",
                        all.x = TRUE)

write.csv(source_metrics,
          "04_Results/04_Source_Impact.csv",
          row.names = FALSE)
```

3.5 Most Prolific Authors

```
#| output: false
# Author productivity analysis
author_productivity <- authorProdOverTime(M, k = 15, graph = FALSE)

# Most relevant authors (by article count)
most_relevant_authors <- M %>%
  # Split multiple authors
  mutate(authors = strsplit(as.character(AU), ";")) %>%
  unnest(authors) %>%
  mutate(authors = trimws(authors)) %>%
  group_by(authors) %>%
  summarise(
    Articles = n(),
    First_Year = min(PY, na.rm = TRUE),
    Last_Year = max(PY, na.rm = TRUE),
    Total_Citations = sum(TC, na.rm = TRUE)
  ) %>%
  arrange(desc(Articles)) %>%
  mutate(Rank = row_number())

top_authors <- head(most_relevant_authors, 20)
```

```
write.csv(top_authors,
          "04_Results/05_Most_Relevant_Authors.csv",
          row.names = FALSE)
```

3.6 Author Impact Metrics

```
#| output: false
# Calculate h-index, g-index, m-index for authors
author_impact <- Hindex(M, field = "author", elements = top_authors$authors,
                        sep = ";", years = 10)

# Get detailed metrics
author_metrics <- author_impact$H %>%
  mutate(
    `m-index` = round(`h-index` / (2024 - `Production Start` + 1), 3),
    Career_Length = 2024 - `Production Start` + 1
  )

write.csv(author_metrics,
          "04_Results/06_Authors_Impact.csv",
          row.names = FALSE)

# Author dominance ranking
dominance <- dominance(results, k = 10)

write.csv(dominance$dfPY,
          "04_Results/07_Author_Dominance.csv",
          row.names = FALSE)
```

3.7 Most Cited Documents

```
#| output: false
# Most cited documents globally
most_cited_docs <- M %>%
  arrange(desc(TC)) %>%
  select(AU, PY, TI, SO, TC, DI) %>%
  head(20) %>%
  mutate(Rank = row_number())

write.csv(most_cited_docs,
          "04_Results/08_Most_Cited_Documents.csv",
          row.names = FALSE)

# Most cited references
```

```
cited_refs <- citations(M, field = "article", sep = ";")
top_cited_refs <- as.data.frame(cited_refs$Cited) %>%
  head(20) %>%
  mutate(Rank = row_number())

names(top_cited_refs) <- c("Reference", "Citations", "Rank")

write.csv(top_cited_refs,
          "04_Results/09_Most_Cited_References.csv",
          row.names = FALSE)
```

3.8 Country Analysis

```
#| output: false
# Extract countries from affiliations
country_production <- metaTagExtraction(M, Field = "AU_CO", sep = ";")

# Country scientific production
country_stats <- country_production %>%
  mutate(countries = strsplit(as.character(AU_CO), ";")) %>%
  unnest(countries) %>%
  mutate(countries = trimws(countries)) %>%
  filter(!is.na(countries) & countries != "") %>%
  group_by(countries) %>%
  summarise(
    Articles = n(),
    Total_Citations = sum(TC, na.rm = TRUE),
    Avg_Citations = round(mean(TC, na.rm = TRUE), 2)
  ) %>%
  arrange(desc(Articles)) %>%
  mutate(Rank = row_number())

top_countries <- head(country_stats, 20)

write.csv(top_countries,
          "04_Results/10_Country_Production.csv",
          row.names = FALSE)

# Calculate SCP (Single Country Publications) and MCP (Multi Country Publications)
country_collab <- metaTagExtraction(M, Field = "AU_CO", sep = ";")
scp_mcp <- cocMatrix(country_collab, Field = "AU_CO", sep = ";", binary = FALSE)

# This requires more complex calculation - simplified version:
country_collab_stats <- country_production %>%
  mutate(
```

```
    country_count = str_count(AU_CO, ";") + 1,
    is_SCP = country_count == 1,
    is_MCP = country_count > 1
  ) %>%
  group_by(AU_CO) %>%
  summarise(
    SCP = sum(is_SCP),
    MCP = sum(is_MCP),
    Total = n()
  )

# Save
write.csv(country_collab_stats,
          "04_Results/11_Country_Collaboration.csv",
          row.names = FALSE)
```

3.9 Keyword Analysis - CRITICAL FOR STBM

```
#| output: false
# Author Keywords (DE)
author_keywords <- M %>%
  filter(!is.na(DE) & DE != "") %>%
  mutate(keywords = strsplit(as.character(DE), ";")) %>%
  unnest(keywords) %>%
  mutate(keywords = trimws(tolower(keywords))) %>%
  group_by(keywords) %>%
  summarise(
    Occurrences = n(),
    Percentage = round(100 * n() / nrow(M), 2)
  ) %>%
  arrange(desc(Occurrences)) %>%
  mutate(Rank = row_number())

top_author_keywords <- head(author_keywords, 50)

write.csv(top_author_keywords,
          "04_Results/12_Authors_Keywords.csv",
          row.names = FALSE)

# Keywords Plus (ID) - assigned by database
keywords_plus <- M %>%
  filter(!is.na(ID) & ID != "") %>%
  mutate(keywords = strsplit(as.character(ID), ";")) %>%
  unnest(keywords) %>%
  mutate(keywords = trimws(tolower(keywords))) %>%
```

```
  group_by(keywords) %>%
  summarise(
    Occurrences = n(),
    Percentage = round(100 * n() / nrow(M), 2)
  ) %>%
  arrange(desc(Occurrences)) %>%
  mutate(Rank = row_number())

top_keywords_plus <- head(keywords_plus, 50)

write.csv(top_keywords_plus,
          "04_Results/13_Keywords_Plus.csv",
          row.names = FALSE)
```

3.10 Trending Topics Analysis

```
#| output: false
# Field trends over time
trends <- fieldByYear(M, field = "DE", timespan = c(2006, 2024),
                      min.freq = 2, n.items = 5, graph = FALSE)

# Extract trend data
trend_topics <- trends$df

write.csv(trend_topics,
          "04_Results/14_Trend_Topics.csv",
          row.names = FALSE)

# Keyword growth analysis
keyword_growth <- M %>%
  filter(!is.na(DE) & DE != "" & !is.na(PY)) %>%
  mutate(keywords = strsplit(as.character(DE), ";")) %>%
  unnest(keywords) %>%
  mutate(keywords = trimws(tolower(keywords))) %>%
  group_by(PY, keywords) %>%
  summarise(Occurrences = n(), .groups = "drop") %>%
  arrange(PY, desc(Occurrences))

write.csv(keyword_growth,
          "04_Results/15_Keyword_Growth_by_Year.csv",
          row.names = FALSE)
```

STAGE 4: THEMATIC MAPPING (STBM CORE)

4.1 Co-word Analysis - Keyword Co-occurrence Network

```
#| output: false
# Create co-occurrence matrix for author keywords
NetMatrix <- biblioNetwork(M, analysis = "co-occurrences",
                           network = "keywords", sep = ";")

# Network statistics
net_stats <- networkStat(NetMatrix)

# Save network statistics
sink("04_Results/16_Coword_Network_Statistics.txt")
print(summary(net_stats))
sink()

# Normalize matrix
NetMatrix_norm <- normalizeSimilarity(NetMatrix, type = "association")

# Calculate keyword correlations
keyword_corr <- cor(as.matrix(NetMatrix_norm), method = "pearson")

# Extract top correlations
top_correlations <- which(keyword_corr > 0.3 & keyword_corr < 1, arr.ind = TRUE)
corr_df <- data.frame(
  Keyword1 = rownames(keyword_corr)[top_correlations[,1]],
  Keyword2 = colnames(keyword_corr)[top_correlations[,2]],
  Correlation = keyword_corr[top_correlations]
) %>%
  filter(Keyword1 < Keyword2) %>%  # Remove duplicates
  arrange(desc(Correlation))

write.csv(corr_df,
          "04_Results/17_Keyword_Correlations.csv",
          row.names = FALSE)
```

4.2 Strategic Diagram (Thematic Map) - STBM SIGNATURE

```
#| output: false
# Generate thematic map
# This is the core of STBM - positioning themes by centrality and density

thematic_map <- thematicMap(M, field = "DE", n = 250,
                            minfreq = 3, stemming = FALSE,
                            size = 0.5, n.labels = 5, repel = TRUE)
```

```
# Extract cluster data
clusters <- thematic_map$clusters

# Save cluster assignments
cluster_keywords <- clusters %>%
  select(cluster_label, words, Freq, Centrality, Density, Quadrant) %>%
  arrange(cluster_label)

write.csv(cluster_keywords,
          "04_Results/18_Thematic_Clusters.csv",
          row.names = FALSE)

# Quadrant classification
quadrant_summary <- clusters %>%
  group_by(Quadrant) %>%
  summarise(
    Themes = n(),
    Avg_Centrality = mean(Centrality),
    Avg_Density = mean(Density),
    Total_Words = sum(Freq)
  )

quadrant_labels <- data.frame(
  Quadrant = c(1, 2, 3, 4),
  Label = c("Motor Themes (High Cent, High Dens)",
            "Niche Themes (Low Cent, High Dens)",
            "Emerging/Declining (Low Cent, Low Dens)",
            "Basic Themes (High Cent, Low Dens)")
)

quadrant_summary <- merge(quadrant_summary, quadrant_labels, by = "Quadrant")

write.csv(quadrant_summary,
          "04_Results/19_Strategic_Diagram_Quadrants.csv",
          row.names = FALSE)

# Plot strategic diagram
png("04_Results/strategic_diagram.png", width = 3000, height = 2400, res = 300)
plot(thematic_map$map)
dev.off()
```

4.3 Thematic Evolution Analysis

```
#| output: false
# Divide timespan into periods
# Adjust periods based on your data distribution
periods <- c(2006, 2015, 2020, 2024)

# Split dataset by periods
M_periods <- list()
for(i in 1:(length(periods)-1)) {
  M_periods[[i]] <- M %>%
    filter(PY >= periods[i] & PY < periods[i+1])

  cat("Period", i, ":", periods[i], "-", periods[i+1]-1,
      "- Documents:", nrow(M_periods[[i]]), "\n")
}

# Create thematic evolution
years <- c("2006-2014", "2015-2019", "2020-2024")

nexus <- thematicEvolution(M, field = "DE", years = years,
                           n = 100, minFreq = 2)

# Save evolution data
evolution_data <- nexus$Data

write.csv(evolution_data,
          "04_Results/20_Thematic_Evolution.csv",
          row.names = FALSE)

# Plot evolution
png("04_Results/thematic_evolution.png", width = 3600, height = 2400, res = 300)
plotThematicEvolution(nexus$Nodes, nexus$Edges)
dev.off()

# Sankey diagram of theme evolution
# Requires additional libraries
library(networkD3)

sankeyPlot <- plotThematicEvolution(nexus$Nodes, nexus$Edges,
                                    measure = "inclusion",
                                    min.flow = 2)

# Save Sankey
saveNetwork(sankeyPlot, "04_Results/thematic_evolution_sankey.html")
```

4.4 Conceptual Structure (Factorial Analysis)

```
#| output: false

# Conceptual structure using Multiple Correspondence Analysis (MCA)
CS <- conceptualStructure(M, field = "DE", method = "MCA",
                          minDegree = 3, clust = 5, stemming = FALSE,
                          labelsize = 10, documents = 20)

# Extract cluster assignments
concept_clusters <- as.data.frame(CS$km.res$cluster)
concept_clusters$Keyword <- rownames(concept_clusters)
names(concept_clusters) <- c("Cluster", "Keyword")

write.csv(concept_clusters,
          "04_Results/21_Conceptual_Clusters_MCA.csv",
          row.names = FALSE)

# Save factorial map
png("04_Results/conceptual_structure_map.png", width = 3000, height = 2400, res = 300)
plot(CS$graph_terms)
dev.off()

# Document clustering based on keywords
png("04_Results/document_clustering.png", width = 3000, height = 2400, res = 300)
plot(CS$graph_documents_Km)
dev.off()
```

STAGE 5: SEMANTIC ANALYSIS (STBM UNIQUE COMPONENT)

5.1 Semantic Field Identification

This is qualitative interpretation of quantitative patterns

```
#| output: false
# Extract all unique keywords for semantic coding
all_keywords <- unique(c(
  author_keywords$keywords,
  keywords_plus$keywords
))

# Create semantic field coding framework
semantic_fields <- data.frame(
  Keyword = all_keywords,
  Semantic_Field = NA,
```

```
  Domain = NA,
  Paradigm = NA,
  Notes = NA
)

# Manual coding process (researcher judgment)
# Example coding rules:

code_semantic_fields <- function(keyword) {
  keyword_lower <- tolower(trimws(keyword))

  # Environmental-Scientific Domain
  if(grepl("light pollution|dark sky|night sky|atmospheric|conservation|protected area|envi
    return(list(
      Field = "Environmental-Scientific",
      Domain = "Conservation Science",
      Paradigm = "Conservation"
    ))
  }

  # Tourism-Commercial Domain
  if(grepl("tourism|tourist|visitor|destination|marketing|experience|satisfaction|hotel|attr
    return(list(
      Field = "Tourism-Commercial",
      Domain = "Tourism Studies",
      Paradigm = "Development"
    ))
  }

  # Sustainability-Development Domain
  if(grepl("sustain|rural development|community|stakeholder|impact|development", keyword_low
    return(list(
      Field = "Sustainability-Development",
      Domain = "Sustainable Development",
      Paradigm = "Sustainability"
    ))
  }

  # Heritage-Cultural Domain
  if(grepl("heritage|cultural|authenticit|indigenous|interpretation|identity|meaning", keywo
    return(list(
      Field = "Heritage-Cultural",
      Domain = "Heritage Studies",
      Paradigm = "Cultural"
    ))
```

```
  }

  # Governance-Policy Domain
  if(grepl("policy|governance|regulation|certification|designation|management|planning", key
    return(list(
      Field = "Governance-Policy",
      Domain = "Public Policy",
      Paradigm = "Governance"
    ))
  }

  # Default
  return(list(
    Field = "Other",
    Domain = "Unclassified",
    Paradigm = "Mixed"
  ))
}

# Apply coding
for(i in 1:nrow(semantic_fields)) {
  coding <- code_semantic_fields(semantic_fields$Keyword[i])
  semantic_fields$Semantic_Field[i] <- coding$Field
  semantic_fields$Domain[i] <- coding$Domain
  semantic_fields$Paradigm[i] <- coding$Paradigm
}

write.csv(semantic_fields,
          "04_Results/22_Semantic_Field_Coding.csv",
          row.names = FALSE)

# Aggregate semantic field statistics
semantic_stats <- semantic_fields %>%
  group_by(Semantic_Field) %>%
  summarise(
    Keywords = n(),
    Percentage = round(100 * n() / nrow(semantic_fields), 2)
  ) %>%
  arrange(desc(Keywords))

write.csv(semantic_stats,
          "04_Results/23_Semantic_Field_Statistics.csv",
          row.names = FALSE)
```

5.2 Paradigm Analysis

```
#| output: false
# Identify which paradigms dominate in different time periods
paradigm_evolution <- M %>%
  filter(!is.na(DE) & DE != "") %>%
  mutate(keywords = strsplit(as.character(DE), ";")) %>%
  unnest(keywords) %>%
  mutate(keywords = trimws(tolower(keywords))) %>%
  left_join(semantic_fields, by = c("keywords" = "Keyword")) %>%
  group_by(PY, Paradigm) %>%
  summarise(Count = n(), .groups = "drop") %>%
  filter(!is.na(Paradigm))

# Calculate paradigm proportions by year
paradigm_proportions <- paradigm_evolution %>%
  group_by(PY) %>%
  mutate(
    Total = sum(Count),
    Proportion = round(100 * Count / Total, 2)
  )

write.csv(paradigm_proportions,
          "04_Results/24_Paradigm_Evolution.csv",
          row.names = FALSE)

# Visualize paradigm shifts
library(ggplot2)

ggplot(paradigm_proportions, aes(x = PY, y = Proportion, fill = Paradigm)) +
  geom_area(position = "stack", alpha = 0.7) +
  labs(title = "Paradigmatic Evolution Over Time",
       x = "Year",
       y = "Proportion (%)",
       fill = "Paradigm") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, size = 14, face = "bold"))

ggsave("04_Results/paradigm_evolution_plot.png", width = 12, height = 7, dpi = 300)
```

5.3 Semantic Integration Analysis

```
#| output: false
# Analyze which semantic fields co-occur (integration patterns)
semantic_cooccurrence <- M %>%
```

```
  filter(!is.na(DE) & DE != "") %>%
  select(SR, DE) %>%  # SR = unique identifier
  mutate(keywords = strsplit(as.character(DE), ";")) %>%
  unnest(keywords) %>%
  mutate(keywords = trimws(tolower(keywords))) %>%
  left_join(semantic_fields, by = c("keywords" = "Keyword")) %>%
  filter(!is.na(Semantic_Field)) %>%
  group_by(SR) %>%
  summarise(
    Fields = paste(unique(Semantic_Field), collapse = "; "),
    Field_Count = length(unique(Semantic_Field))
  )

# Calculate integration patterns
integration_patterns <- semantic_cooccurrence %>%
  group_by(Fields) %>%
  summarise(Documents = n()) %>%
  arrange(desc(Documents)) %>%
  head(20)

write.csv(integration_patterns,
          "04_Results/25_Semantic_Integration_Patterns.csv",
          row.names = FALSE)

# Calculate integration index (average fields per document)
integration_index <- mean(semantic_cooccurrence$Field_Count)

cat("Semantic Integration Index:", round(integration_index, 2),
    "fields per document\n")
```

STAGE 6: NETWORK VISUALIZATION WITH VOSVIEWER

6.1 Prepare Data for VOSviewer

VOSviewer provides superior visualization compared to R for large networks.

Steps in VOSviewer:

1. **Launch VOSviewer**
   - Download from: https://www.vosviewer.com/
   - No installation required (runs from folder)
2. **Create Network from Bibliographic Data**

   Option A: Direct Import

- File → Create → Create a map based on bibliographic data
- Read data from: "Bibliographic database files"
- Choose: RIS, BibTeX, or tab-delimited file
- Select file: `for_vosviewer.ris`
- Type of analysis: Co-occurrence
- Unit of analysis: Author keywords
- Counting method: Full counting
- Minimum number of occurrences: 3-5 (adjust based on dataset size)
- Click "Finish"

Option B: Create from Matrix (Advanced)

- Create co-occurrence matrix in R (done in Phase 4.1)
- Export matrix to CSV
- VOSviewer → Create map from matrix

3. **Customize Visualization**

   Network View:

   - Items → Show/hide items → Show items with X links
   - Visualization → Layout → Attraction/Repulsion (adjust spacing)
   - Visualization → Colors → Based on clusters / based on scores

   Overlay Visualization (Temporal):

   - Visualization → Overlay visualization
   - Score: Avg. pub. year (shows temporal evolution)
   - Color scheme: Rainbow (blue = old, red = new)

   Density Visualization:

   - Visualization → Density visualization
   - Shows research hotspots

4. **Export High-Quality Images**

   - File → Save map
   - File → Export → Export to PNG (for publications)
   - Resolution: 300 DPI minimum
   - Size: 3000 x 2000 pixels recommended

5. **Export Network Data**

   - File → Export → Export to Pajek
   - File → Export → Export to network text files
   - Can then import to Gephi for advanced visualization

6.2 Multiple Network Types in VOSviewer

Create separate networks for different analyses:

Network 1: Author Keywords Co-occurrence - Unit: Author keywords - Purpose: Thematic structure - Minimum occurrences: 3 - Save as: `keyword_network.vosviewer`

Network 2: Co-authorship Network - Type: Co-authorship - Unit: Authors - Purpose: Collaboration structure - Minimum documents: 2 - Save as: `coauthorship_network.vosviewer`

Network 3: Co-citation Network - Type: Co-citation - Unit: Cited references - Purpose: Intellectual structure - Minimum citations: 3 - Save as: `cocitation_network.vosviewer`

Network 4: Bibliographic Coupling - Type: Bibliographic coupling - Unit: Documents - Purpose: Research fronts - Minimum citations: 5 - Save as: `bibcoupling_network.vosviewer`

Network 5: Country Collaboration - Type: Co-authorship - Unit: Countries - Purpose: Geographic collaboration - Minimum documents: 2 - Save as: `country_network.vosviewer`

STAGE 7: ADVANCED ANALYSIS AND SYNTHESIS

7.1 Research Front Identification

```
#| output: false
# Identify research fronts using bibliographic coupling
# Research fronts = clusters of recent papers citing similar literature

recent_papers <- M %>%
  filter(PY >= 2020)  # Focus on recent 5 years

# Create bibliographic coupling network for recent papers
BC_matrix <- biblioNetwork(recent_papers, analysis = "coupling",
                           network = "references", sep = ";")

# Detect communities (research fronts)
library(igraph)

BC_graph <- graph_from_adjacency_matrix(BC_matrix,
                                        mode = "undirected",
                                        weighted = TRUE)

# Community detection
communities <- cluster_louvain(BC_graph)

# Extract community assignments
community_membership <- data.frame(
  Document = V(BC_graph)$name,
```

```
  Community = communities$membership
)

# Match back to original data
M_recent_communities <- recent_papers %>%
  left_join(community_membership, by = c("SR" = "Document"))

# Characterize each research front by dominant keywords
research_fronts <- M_recent_communities %>%
  filter(!is.na(Community) & !is.na(DE)) %>%
  group_by(Community) %>%
  mutate(keywords = strsplit(as.character(DE), ";")) %>%
  unnest(keywords) %>%
  mutate(keywords = trimws(tolower(keywords))) %>%
  group_by(Community, keywords) %>%
  summarise(Freq = n(), .groups = "drop") %>%
  group_by(Community) %>%
  arrange(Community, desc(Freq)) %>%
  slice_head(n = 10)

write.csv(research_fronts,
          "04_Results/26_Research_Fronts.csv",
          row.names = FALSE)
```

7.2 Knowledge Gap Identification

```
#| output: false
# Identify gaps by analyzing:
# 1. Basic themes (high centrality, low density)
# 2. Keywords with few connections
# 3. Unexplored keyword combinations

# Gaps from strategic diagram
basic_themes <- cluster_keywords %>%
  filter(Quadrant == 4) %>%  # Quadrant 4 = Basic themes
  select(cluster_label, words, Centrality, Density)

write.csv(basic_themes,
          "04_Results/27_Knowledge_Gaps_Basic_Themes.csv",
          row.names = FALSE)

# Low-connectivity keywords (potential gaps)
keyword_network_stats <- data.frame(
  Keyword = rownames(NetMatrix),
  Degree = rowSums(NetMatrix > 0),  # Number of connections
```

```
  Strength = rowSums(NetMatrix),     # Weighted connections
  Occurrences = diag(NetMatrix)
) %>%
  mutate(
    Avg_Connection_Strength = Strength / Degree,
    Isolation_Index = Occurrences / Degree  # High = isolated topic
  ) %>%
  arrange(desc(Isolation_Index))

# High-occurrence but low-connectivity keywords = undertheorized gaps
gaps_isolated <- keyword_network_stats %>%
  filter(Occurrences >= 5 & Degree < 5)

write.csv(gaps_isolated,
          "04_Results/28_Knowledge_Gaps_Isolated_Keywords.csv",
          row.names = FALSE)
```

7.3 Future Directions Projection

```
#| output: false
# Identify emerging themes using:
# 1. Recent keyword growth
# 2. Low current frequency but high growth rate
# 3. Quadrant 3 themes moving toward higher density/centrality

# Calculate keyword growth rates
keyword_trends <- M %>%
  filter(!is.na(DE) & !is.na(PY)) %>%
  mutate(
    Period = case_when(
      PY <= 2015 ~ "Early",
      PY <= 2020 ~ "Middle",
      PY >= 2021 ~ "Recent"
    ),
    keywords = strsplit(as.character(DE), ";")
  ) %>%
  unnest(keywords) %>%
  mutate(keywords = trimws(tolower(keywords))) %>%
  group_by(Period, keywords) %>%
  summarise(Count = n(), .groups = "drop") %>%
  pivot_wider(names_from = Period, values_from = Count, values_fill = 0) %>%
  mutate(
    Total = Early + Middle + Recent,
    Growth_Score = (Recent - Early) / (Early + 1),  # Avoid division by zero
    Acceleration = Recent / (Middle + 1)
```

```
  ) %>%
  filter(Total >= 3) %>%
  arrange(desc(Growth_Score))

emerging_keywords <- keyword_trends %>%
  filter(Growth_Score > 1.5 | Acceleration > 2) %>%
  head(20)

write.csv(emerging_keywords,
          "04_Results/29_Emerging_Keywords.csv",
          row.names = FALSE)

# Declining keywords (for context)
declining_keywords <- keyword_trends %>%
  filter(Growth_Score < -0.5) %>%
  arrange(Growth_Score) %>%
  head(20)

write.csv(declining_keywords,
          "04_Results/30_Declining_Keywords.csv",
          row.names = FALSE)
```

STAGE 8: STBM SYNTHESIS AND REPORTING

8.1 Create STBM Integration Matrix

```
#| output: false
# Integrate all dimensions into comprehensive overview

stbm_synthesis <- list(

  # Dimension 1: Performance Structure
  performance = list(
    total_documents = nrow(M),
    timespan = paste(min(M$PY), max(M$PY), sep = "-"),
    sources = length(unique(M$SO)),
    authors = length(unique(unlist(strsplit(M$AU, ";")))),
    growth_rate = mean(annual_production$Growth_Rate, na.rm = TRUE),
    top_source = top_sources$SO[1],
    top_author = top_authors$authors[1],
    top_country = top_countries$countries[1]
  ),

  # Dimension 2: Intellectual Structure
  intellectual = list(
```

```
    most_cited_paper = most_cited_docs$TI[1],
    citation_classics = nrow(most_cited_docs),
    h_index_field = max(author_impact$H)
  )
)
```

APPENDIX: TROUBLESHOOTING GUIDE

Common Issues and Solutions

Issue 1: Import Errors from Different Databases

```
#| output: false
# Solution: Manual format conversion
# If automatic conversion fails, manually map columns

manual_convert <- function(file_path, source_db) {

  if(source_db == "wos") {
    data <- readLines(file_path)
    # Custom parsing for WoS format
    # ... (database-specific code)
  }

  if(source_db == "scopus") {
    data <- read.csv(file_path, stringsAsFactors = FALSE)
    # Map Scopus columns to standard format
    # ... (database-specific code)
  }

  return(standardized_data)
}
```

Issue 2: Encoding Problems (Special Characters)

```
#| output: false
# Solution: Force UTF-8 encoding
M$TI <- iconv(M$TI, from = "latin1", to = "UTF-8", sub = "")
M$AB <- iconv(M$AB, from = "latin1", to = "UTF-8", sub = "")
```

Issue 3: Memory Issues with Large Datasets

```
#| output: false
# Solution: Process in chunks or increase memory
options(java.parameters = "-Xmx8g")  # Increase to 8GB RAM
gc()  # Garbage collection to free memory
```

Issue 4: VOSviewer Won't Open File

```
Solution:
1. Ensure file encoding is UTF-8
2. Check file size (<100MB for best performance)
3. Try RIS format instead of tab-delimited
4. Verify no special characters in paths
```

Issue 5: Thematic Map Returns Error

```
#| output: false
# Solution: Adjust parameters
# If error: "not enough keywords"
thematic_map <- thematicMap(M, field = "DE",
                            minfreq = 2,  # Lower threshold
                            n = 500)      # Increase n

# If error: "clustering failed"
thematic_map <- thematicMap(M, field = "DE",
                            minfreq = 3,
                            size = 0.7,   # Adjust size
                            repel = FALSE) # Disable repel
```

STBM Quality Criteria

Checklist for High-Quality STBM Analysis

- ☐ **Data Retrieval**
  - ☐ Multiple databases searched (WoS, Scopus, OpenAlex minimum)
  - ☐ Search strategy documented and replicable
  - ☐ Boolean operators correctly applied
  - ☐ Timespan justified
  - ☐ Document type filters appropriate
- ☐ **Data Processing**
  - ☐ Automatic deduplication completed
  - ☐ Manual deduplication verification performed
  - ☐ Data quality report generated
  - ☐ Missing data addressed or documented
  - ☐ Final dataset >100 documents (minimum for robust analysis)
- ☐ **Bibliometric Analysis**
  - ☐ Main information table complete
  - ☐ Temporal analysis covers full timespan
  - ☐ Top sources, authors, countries identified
  - ☐ Citation metrics calculated
  - ☐ Collaboration patterns analyzed
- ☐ **Thematic Mapping**
  - ☐ Co-word network constructed
  - ☐ Strategic diagram generated with clear quadrants
  - ☐ Thematic evolution analyzed across periods

- ☐ Conceptual structure mapped
- ☐ Research fronts identified

☐ **Semantic Analysis** (STBM Distinctive)
- ☐ Semantic fields systematically identified
- ☐ Paradigms explicitly analyzed
- ☐ Integration patterns examined
- ☐ Qualitative interpretation documented
- ☐ Field boundaries discussed

☐ **Visualization**
- ☐ High-quality figures generated (300 DPI minimum)
- ☐ VOSviewer networks created
- ☐ Publication-ready tables formatted
- ☐ All visualizations properly labeled

☐ **Synthesis**
- ☐ All dimensions integrated
- ☐ Research gaps clearly identified
- ☐ Future directions proposed
- ☐ Field maturity assessed
- ☐ Practical implications articulated

☐ **Documentation**
- ☐ Reproducibility information saved
- ☐ All scripts commented
- ☐ File inventory maintained
- ☐ Validation report completed

References

- Aria, M., & Cuccurullo, C. (2017). bibliometrix: An R-tool for comprehensive science mapping analysis. *Journal of Informetrics*, *11*(4), 959-975.
- Callon, M., Courtial, J. P., & Laville, F. (1991). Co-word analysis as a tool for describing the network of interactions between basic and technological research. *Scientometrics*, *22*(1), 155-205.
- Cobo, M. J., López-Herrera, A. G., Herrera-Viedma, E., & Herrera, F. (2011). Science mapping software tools: Review, analysis, and cooperative study among tools. *Journal of the American Society for Information Science and Technology*, *62*(7), 1382-1402.
- Cobo, M. J., López-Herrera, A. G., Herrera-Viedma, E., & Herrera, F. (2012). SciMAT: A new science mapping analysis software tool. *Journal of the American Society for Information Science and Technology*, *63*(8), 1609-1630.
- Hirsch, J. E. (2005). An index to quantify an individual's scientific research output. *Proceedings of the National Academy of Sciences*, *102*(46), 16569-16572.

- Van Eck, N. J., & Waltman, L. (2010). Software survey: VOSviewer, a computer program for bibliometric mapping. *Scientometrics*, *84*(2), 523-538.

This protocol is designed to be self-contained and replicable. Users should be able to conduct a complete STBM analysis from start to finish following these steps.